

Symfony 6

Liška

Obsah

Symfony 6.....	1
1. Příprava prostředí:.....	1
5. Vytvoření controlleru.....	2
Response.....	3
Response s parametrem.....	4
X. Instalace základních balíčků:.....	5
X. Přidání balíčku Bootstrap:.....	6
X. Přidání další stránky:.....	6
X. Nastavení databáze:.....	6
X. Tvorba entity:.....	7
X. Tvorba formuláře:.....	11
X. Zbylé CRUD operace:.....	13
X. Generování PDF:.....	18
X. Tvorba přihlášení:.....	19

1. Příprava prostředí:

- nainstalovaný PHP 8.1 nebo vyšší (například xampp s php8)
- nainstalujte Composer, který se používá k instalaci PHP balíčků (pro všechny uživatele – nainstaluje se do c:\composer).
- Volitelně nainstalovat Symfony CLI nebo raději si stáhnout soubor symfony.exe (je na disku S...smrcka\wt, případně v této lekci a tento soubor vložit do složky c:\composer

Kontrola cest: pokud zobrazíte systémové proměnné příkazem set v proměnné path bude c:\composer a take pro xampp c:\xampp\php\

Pro urychlení odblokovat v php.ini

extension=zip

Nový project doporučuji dělat ve verzi 6.4, která je podporovaná

```
Path=C:\Program Files\Git\cmd;C:\Program Files (x86)\Eclipse Adoptium\jre-8.0.382.5-hotspot\bin;C:\composer;c:\xampp\php\;
```

2. Pro Symfony CLI otevřete svůj konzolový terminál:

symfony new my_project_directory --version="6.4.*" --webapp

2. Pro Composer otevřete svůj konzolový terminál:

composer create-project symfony/skeleton:"6.4.*" my_project_directory

Následně se přesuňte do projektu přes příkazy:

cd my_project_directory

composer require webapp

composer je příkaz pro spuštění správce závislostí.

require je příkaz, který přidává novou závislost do vašeho projektu.

webapp je název balíčku nebo knihovny, kterou chcete přidat.

3. Ať už spustíte jakýkoli příkaz k vytvoření Symfony aplikace, všechny z nich vytvoří nový adresář `my_project_directory/`, stáhnou do něj některé závislosti a dokonce vygenerují základní adresáře a soubory, které budete potřebovat k zahájení práce.

4. *Spuštění a test projektu:*

Pro spuštění:

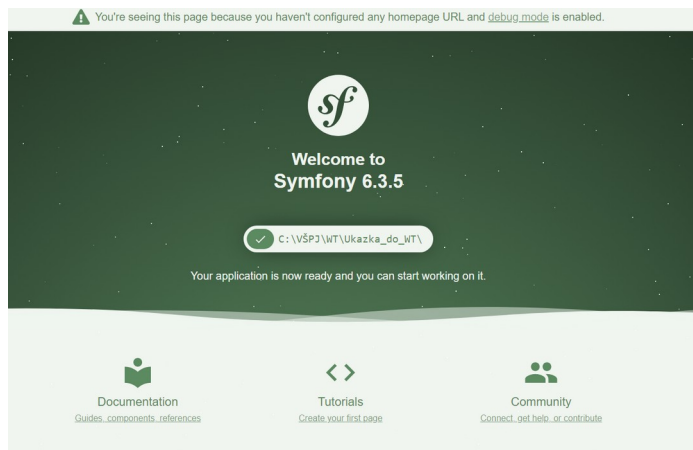
symfony server:start -d

Pro zastavení:

symfony server:stop

5. Vytvoření controlleru

- Nenastavená Homepage



-

symfony console make:controller HomeController

nebo

php bin/console make:controller HomeController

```
class HomeController extends AbstractController
{
    #[Route('/', name: 'app_home')]
    public function index(): Response
    {
        return $this->render('home/index.html.twig', [
            'controller_name' => 'HomeController',
        ]);
    }
}
```

- Definujeme cestu na Homepage v **config/routes.yaml** a přidáme

```
controllers:
    resource:
        path: ../src/Controller/
        namespace: App\Controller
        type: attribute
    home:
        path: /
        controller: App\Controller\HomeController::index
```

-

- Nová domovská stránka v

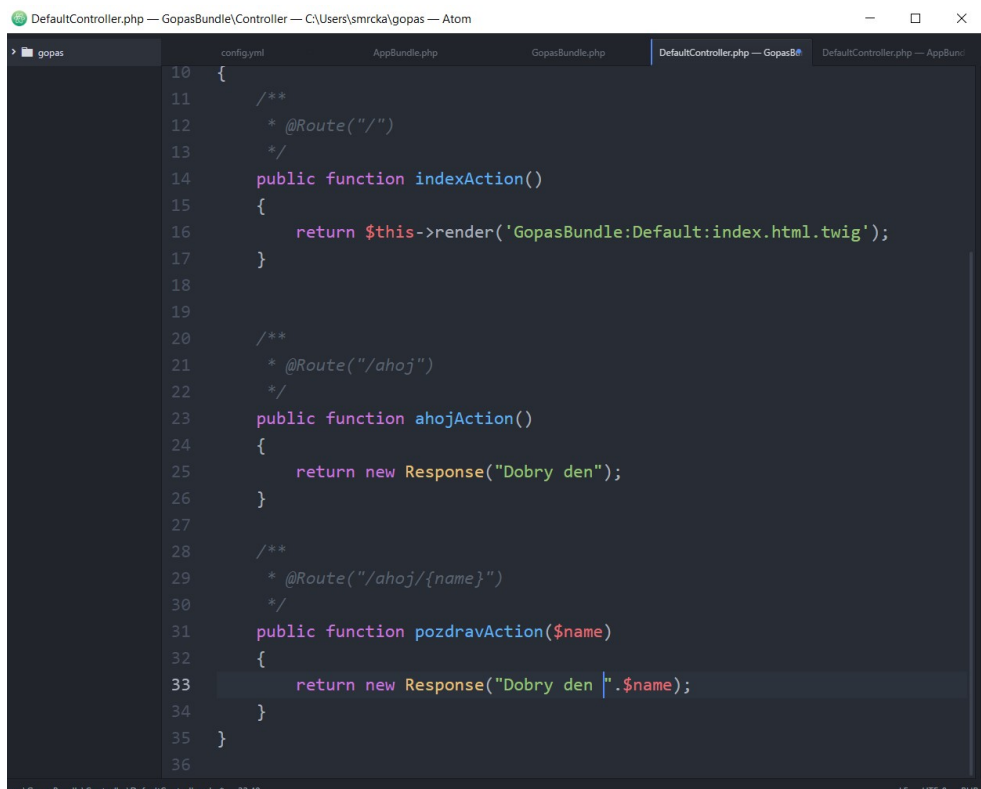
Hello HomeController!

This friendly message is coming from:

- Your controller at [src/Controller/HomeController.php](#)
- Your template at [templates/home/index.html.twig](#)

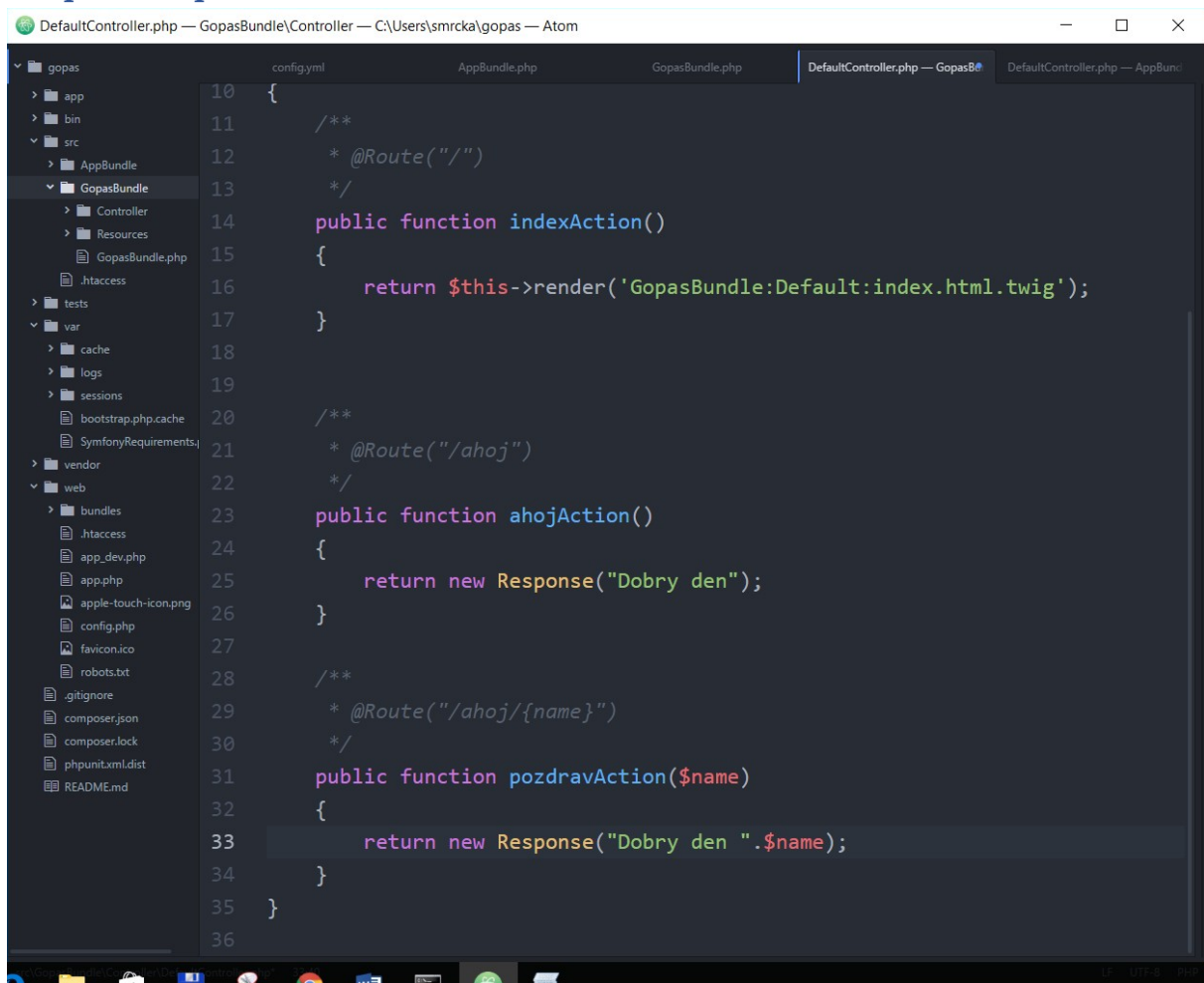
Response

```
use Symfony\Component\HttpFoundation\Response;
```



```
DefaultController.php — GopasBundle\Controller — C:\Users\smrcka\gopas — Atom
> gopas
config.yml  AppBundle.php  GopasBundle.php  DefaultController.php — GopasB...  DefaultController.php — AppBundle...
10 {
11     /**
12      * @Route("/")
13      */
14     public function indexAction()
15     {
16         return $this->render('GopasBundle:Default:index.html.twig');
17     }
18
19     /**
20      * @Route("/ahoj")
21      */
22     public function ahojAction()
23     {
24         return new Response("Dobry den");
25     }
26
27     /**
28      * @Route("/ahoj/{name}")
29      */
30     public function pozdravAction($name)
31     {
32         return new Response("Dobry den | ".$name);
33     }
34 }
35
36
```

Response s parametrem



```
10 {
11     /**
12      * @Route("/")
13      */
14     public function indexAction()
15     {
16         return $this->render('GopasBundle:Default:index.html.twig');
17     }
18
19     /**
20      * @Route("/ahoj")
21      */
22     public function ahojAction()
23     {
24         return new Response("Dobry den");
25     }
26
27     /**
28      * @Route("/ahoj/{name}")
29      */
30     public function pozdravAction($name)
31     {
32         return new Response("Dobry den ".$name);
33     }
34 }
35
36
```

/ahoj/honzo

X. Instalace základních balíčků:

composer require symfony/webpack-encore-bundle

npm install

npm run dev dává chybu

řešení

composer remove symfony/ux-turbo symfony/asset-mapper
symfony/stimulus-bundle

composer require symfony/webpack-encore-bundle symfony/ux-turbo
symfony/stimulus-bundle

npm install

npm run dev

npm run build

X. Přidání balíčku Bootstrap:

npm install bootstrap@4.6.0 --save-dev

změňte koncovku souboru **assets/styles/app.css** to **app.scss**

přidejte do něj **@import "~bootstrap/scss/bootstrap";**

```
Enable File Watcher to compile SCSS to CSS?

1  @import "~bootstrap/scss/bootstrap";
2
3  body {
4    background-color: skyblue;
5  }
6
7  |
```

do souboru **assets/app.js** přidejte **import 'bootstrap';** a **import './styles/app.scss';**

```
app.scss  app.js x
1  import './bootstrap.js';
2  import 'bootstrap';
3  /*
4   * Welcome to your app's main JavaScript file!
5   *
6   * This file will be included onto the page via the importmap() Twig function,
7   * which should already be in your base.html.twig.
8   */
9  import './styles/app.scss';
10
11  console.log('This log comes from assets/app.js - welcome to AssetMapper! 🐘');
12
```

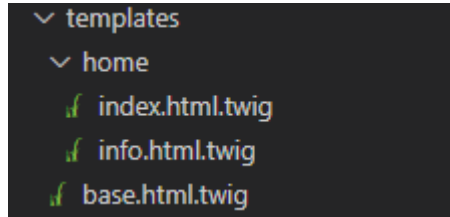
v **webpack.config.js** odkomentujte **.enableSassLoader()**
a **.autoProvidejQuery()**

npm install sass-loader@^16.0.1 sass --save-dev

npm run dev

X. Přidání další stránky:

- Tvorba stránky v **/templates**



- Přidání další metody do **HomeController**

```
#[Route('/info', name: 'app_home')] //Cesta s názvem -> '/info' // Název pro cestu kontroler 'app_home'
public function info(): Response //Název metody
{
    return $this->render('home/info.html.twig', [ //Cesta jakou stránku má kontroler vykreslit
        'controller_name' => 'HomeController',
        'variable_test' => 'Value_test', //Název proměnné a její hodnota
    ]);
}
```

X. Nastavení databáze:

- Otevřeme soubor .env
- DATABASE_URL podle vašeho databázového serveru
- Příklad použití: SQL je **MySQL** a vaše databáze se jmenuje **symfony_db**
DATABASE_URL="mysql://username:password@localhost:3306/symfony_db"
- Změňte username, password a symfony_db podle vašich údajů

Pozn pro xampp například

```
DATABASE_URL="mysql://pokus:pokus@127.0.0.1:3306/pokus?
serverVersion=mariadb-10.4.28"
```

X. Tvorba entity:

symfony console make:entity Product nebo **php bin/console make:entity Product**

V případě, že nejde, je nutné instalovat:

```
$ composer require symfony/orm-pack
$ composer require --dev symfony/maker-bundle
```

Následně můžete definovat Vaši entitu přímo v konzoli nebo dát **ENTER** a nadefinovat si jí celou sami


```

<?php

namespace App\Entity;

use App\Repository\ProductRepository;
use Doctrine\ORM\Mapping as ORM;

#[ORM\Entity(repositoryClass: ProductRepository::class)]
class Product
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
    private ?int $id = null;

    #[ORM\Column(length: 50)]
    private ?string $name = null;

    public function getId(): ?int
    {
        return $this->id;
    }

    public function getName(): ?string
    {
        return $this->name;
    }

    public function setName(string $name): static
    {
        $this->name = $name;

        return $this;
    }
}

```

Sloupce id a name

Tvorba migrace: **php bin/console make:migration**

Provedení migrace: **php bin/console doctrine:migrations:migrate --no-interaction**

V phpmyadmin můžeme přidat data

pak rychle zjistíme, že tam data jsou

php bin/console dbal:run-sql 'SELECT * FROM product'

X. Výpis entity na stránku:

- Přidáme do HomeController cesty k entitě a Doctrineu
-

```
use Doctrine\Persistence\ManagerRegistry; //Přidáno pro manipulaci s entitami
use App\Entity\Product; //Přidáno pro práci s entitou "Product"
```

- Načteme si všechny záznamy do proměnný a tu si necháme vykreslit na vybranou stránku
-

```
#[Route('/home', name: 'app_home')]
public function index(ManagerRegistry $doctrine): Response
{
    // najít všechny produkty
    $products = $doctrine->getManager()->getRepository(Product::class)->findAll();

    return $this->render('home/index.html.twig', [
        'controller_name' => 'HomeController',
        'products' => $products
    ]);
}
```

-
- Upřesnění controlleru

```
<?php

namespace App\Controller;

use Doctrine\Persistence\ManagerRegistry;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;
use
Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use App\Entity\Product;
class HomeController extends AbstractController
{
    #[Route('/home', name: 'home')]
    public function Home(ManagerRegistry $doctrine): Response
    {
        $products=$doctrine->getManager()-
        >getRepository(Product::class)->findAll();
        return $this->render('home.html.twig', ['products'=>$products]);
    }
}
```

-
-
- Na stránce si pak jednotlivé záznamy vypíšeme přes for cyklus

```
{% for product in products %}
  <tr>
    <td> {{product.getId()}} </td>
    <td> {{product.getName()}} </td>
  </tr>
{% endfor %}
```

-
- Twig pak je

```
{% for product in products %}
{{ product.getId() }}
{{ product.getName() }}<br>
{% endfor %}
```

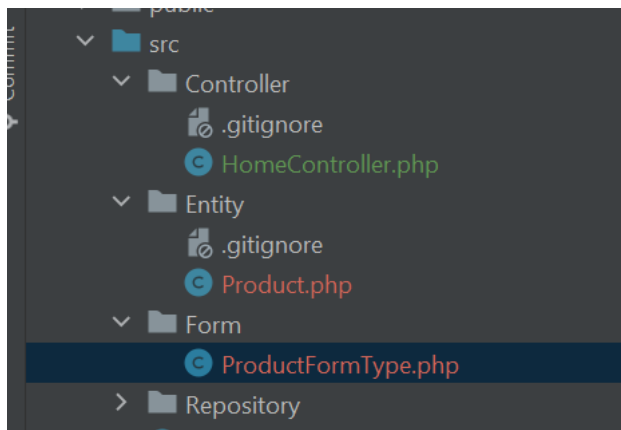
X. Tvorba formuláře:

Vytvoření formuláře:

php bin/console make:form ProductForm

-

Vytvoří se soubor ProductFormType.php



přidejte entitu a volitelně další datové typy (pokud nejsou přidány)

soubor ProductFormType.php

-

```
use App\Entity\Product; //Název entity
use Symfony\Component\Form\AbstractType;
use Symfony\Component\Form\FormBuilderInterface;
use Symfony\Component\OptionsResolver\OptionsResolver;
use Symfony\Component\Form\Extension\Core\Type\TextType; // Přidá další datové typy
```

-

```
class ProductFormType extends AbstractType
{
    public function buildForm(FormBuilderInterface $builder, array $options): void
    {
        $builder
            ->add('name', TextType::class, ['label' => 'Název']); //Specifikuje název pod kterým se to bude zobrazovat
    }

    public function configureOptions(OptionsResolver $resolver): void
    {
        $resolver->setDefaults([
            'data_class' => Product::class,
        ]);
    }
}
```

- **Prakticky**

- `?php`

```
namespace App\Form;
```

```

use App\Entity\Product;
use Symfony\Component\Form\AbstractType;
use Symfony\Component\Form\FormBuilderInterface;
use Symfony\Component\OptionsResolver\OptionsResolver;

class ProductFormType extends AbstractType
{
    public function buildForm(FormBuilderInterface $builder,
array $options): void
    {
        $builder
            ->add('name')
        ;
    }

    public function configureOptions(OptionsResolver
$resolver): void
    {
        $resolver->setDefaults([
            'data_class' => Product::class,
        ]);
    }
}

```

- Následně přidáme další metodu do **HomeController**

```

#[Route('/product/new', name: 'product_new')] //cesta
public function new(Request $request, ManagerRegistry $doctrine): Response
{
    $product = new Product(); //Tvorba nového objektu typu Product

    $form = $this->createForm(ProductFormType::class, $product); //Tvorba nového formuláře podle vzoru ProductFormType

    $form->handleRequest($request); //Předání dat z formuláře

    if ($form->isSubmitted() && $form->isValid()) // Kontrola validity a odeslání
    {
        $product = $form->getData();

        $em = $doctrine->getManager();

        $em->persist($product);
        $em->flush(); //Přidání data do databáze

        return $this->redirect('/'); //Přesměrování na Homepage
    }

    return $this->render('home/product_new.html.twig', [ //Stránka s daným formulářem
        'form' => $form->createView(),
    ]);
}

```

Prakticky HomeController.php

```

namespace App\Controller;

use App\Form\ProductFormType;
use Doctrine\Persistence\ManagerRegistry;
use Symfony\Component\HttpFoundation\Request;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;

```

```
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use App\Entity\Product;
class HomeController extends AbstractController
{
```

.....

```
#[Route('/product/new', name: 'productnew')]
public function new(Request $request, ManagerRegistry $doctrine): Response
{
    $product=new Product();
    $form=$this->createForm(ProductFormType::class,$product);
    $form->handleRequest($request);

    if ($form->isSubmitted() && $form->isValid())
    {

        $products=$form->getData();
        $em=$doctrine->getManager();
        $em->persist($products);
        $em->flush();
        return $this->redirect('/home');
    }

    return $this->render('product_new.html.twig', ['form'=>$form-
>createView()]);
}
```

- `$form = $this->createForm(UserType::class, new User());`: Vytvoří formulář na základě třídy UserType a objektu User.
- `$form->handleRequest($request)`: Zpracuje data z HTTP požadavku a naplní jimi formulář.
- `if ($form->isSubmitted() && $form->isValid()) { ... }`: Zkontroluje, zda byl formulář odeslán a zda je platný. Pokud ano, můžete zde zpracovat data formuláře.
- `return $this->render('user/new.html.twig', ['form' => $form->createView()])`: Vykreslí šablonu new.html.twig a předá do ní pohled formuláře, který se použije k jeho vykreslení v šabloně.

- Vytvoříme stránku **product_new.html.twig**

•

```
{% extends 'base.html.twig' %}

{% block title %}Hello HomeController!{% endblock %}

{% block body %}
<style>
    .example-wrapper { margin: 1em auto; max-width: 800px; width: 95%; font: 18px/1.5 sans-serif; }
    .example-wrapper code { background: #F5F5F5; padding: 2px 6px; }
</style>
{{ form_start(form) }} <!-- Název formuláře jako předáváme v HomeController-->
{{ form_row(form.name) }}
{{ form_row(form.productType) }} <!--Názvy polí podle toho jak jsou definovaná ve ProductFormType-->
<button type="submit">Save</button>
{{ form_end(form) }}

{% endblock %}
```

Prakticky

```
{% block body %}

    {{ form_start(form) }}
    {{ form_row(form.name) }}
    <button type="submit">Save</button>
    {{ form_end(form) }}

{% endblock %}
```

X. Zbylé CRUD operace:

- Přidáme metody podle vybrané operace
- **Edit**
-

```
#[Route('/product/edit/{id}', name: 'product_edit')]
public function edit(Request $request, ManagerRegistry $doctrine, $id): Response //Přidá parametr id pro zjištění záznamu
{
    $product = $doctrine->getManager()->getRepository(Product::class)->find($id); //Najde na základě id záznam
    if (!$product) {
        throw $this->createNotFoundException('No product found for id ' . $id); //Error pokud není záznam nalezen
    }

    $form = $this->createForm(ProductFormType::class, $product); //Tvorba nového formuláře podle vzoru ProductFormType
    $form->handleRequest($request); //Předání dat z formuláře

    if ($form->isSubmitted() && $form->isValid()) {
        $em = $doctrine->getManager(); // Objekt pro práci s entitami

        $em->flush(); // Provedení změn v databázi

        return $this->redirectToRoute('app_home');
    }

    return $this->render('home/product_edit.html.twig', [
        'form' => $form->createView(),
    ]);
}
```

```
Prakticky #[Route('/product/edit/{id}', name: 'product_edit')]
public function edit(Request $request, ManagerRegistry $doctrine, $id):
Response
{
    $product=$doctrine->getManager()->getRepository(Product::class)-
>find($id);
    if (!$product)
    {
        throw $this->createNotFoundException('Produkt s tímto číslem
nenalezen ' . $id);
    }
    $form=$this->createForm(ProductFormType::class,$product);
    $form->handleRequest($request);
    if ($form->isSubmitted() && $form->isValid())
    {

        $em=$doctrine->getManager();
        $em->flush();
        return $this->redirect('/home');
    }

    return $this->render('product_edit.html.twig', ['form'=>$form-
>createView()]);
}
```

- Vytvoříme html stránku na které se bude vytvářet formulář
-

```
<div class="container"> {# Wrap the form in a container to center it #}
  <div class="row justify-content-center"> {# Use Bootstrap to horizontally center the content #}
    <div class="col-md-8"> {# Adjust the column width as needed #}
      <h1>Záznam nového produktu</h1>

      {{ form_start(form, {'attr': {'class': 'form-horizontal'}}) }}
      <div class="form-group">
        {{ form_label(form.name, 'Název', {'label_attr': {'class': 'col-form-label col-sm-2'}}) }}
        <div class="col-sm-10">
          {{ form_widget(form.name, {'attr': {'class': 'form-control'}}) }}
          {{ form_errors(form.name) }}
        </div>
        </div>

        <div class="form-group">
          {{ form_label(form.productType, 'Typ', {'label_attr': {'class': 'col-form-label col-sm-2'}}) }}
          <div class="col-sm-10">
            {{ form_widget(form.productType, {'attr': {'class': 'form-control'}}) }}
            {{ form_errors(form.productType) }}
          </div>
        </div>

        <div class="form-group">
          <div class="col-sm-offset-2 col-sm-10">
            <button type="submit" class="btn btn-primary">Uložit</button>
            <a class="btn btn-primary" href="{{ path('app_home') }}">Zpět</a>
          </div>
        </div>
      {{ form_end(form) }}
    </div>
  </div>
</div>
```

Zjednodušeno

```
% block body %}
  <h1>Oprava produktu</h1>

  {{ form_start(form) }}
  {{ form_widget(form.name) }}
  {{ form_errors(form.name) }}

  <button type="submit">Save</button>
  <a href="{{ path('home') }}">Home</a>
  {{ form_end(form) }}
{% endblock %}
```


- Delete

-

```
#[Route('/product/delete/{id}', name: 'product_delete')]
public function delete(Request $request, ManagerRegistry $doctrine, $id): Response
{
    $product = $doctrine->getManager()->getRepository(Product::class)->find($id); //Najde na základě id záznam

    if (!$product)
    {
        throw $this->createNotFoundException('No product found for id ' . $id); //Error pokud není záznam nalezen
    }

    if ($this->isCsrfTokenValid('delete'.$product->getId(), $request->request->get('_token')))
    {
        $em = $doctrine->getManager();
        $em->remove($product); //Odebere záznam
        $em->flush(); // Provedení změn v databázi

        return $this->render('home/index.html.twig', [ //Cesta jakopu stránku má kontroler vykreslit
            'controller_name' => 'HomeController',
            'variable_test' => 'Value_test', //Název proměně a její hodnota
        ]);
    }

    return $this->render('home/product_delete.html.twig', [
        'product' => $product,
    ]);
}
```

Prakticky

```
#[Route('/product/delete/{id}', name: 'product_delete')]
public function delete(Request $request, ManagerRegistry $doctrine, $id): Response
{
    $product=$doctrine->getManager()->getRepository(Product::class)->find($id);
    if (!$product)
    {
        throw $this->createNotFoundException('Produkt s tímto číslem nenalezen ' . $id);
    }

    if ($this->isCsrfTokenValid('delete'.$product->getId(), $request->request->get('_token')))
    {
        $em=$doctrine->getManager();
        $em->remove($product);
        $em->flush();
        return $this->render('index.html.twig', [
            'controllername'=>"HomeController",
            'variable_test'=>'Value_Test'
        ]);
    }

    return $this->render('delete.html.twig', ['product'=>$product]);
}
```

Poznámka (csrf): <https://www.root.cz/clanky/uvod-do-frameworku-symfony-jednoduche-formulare/>

- Vytvoříme html stránku na které se bude vykreslovat záznam, který budeme chtít smazat
-

```
<div class="row justify-content-center">
  <div class="col-auto">
    <table class="table table-bordered">
      <thead>
        <tr>
          <th scope="col">Id</th>
          <th scope="col">Name</th>
          <th scope="col">Type</th>
        </tr>
      </thead>
      <tbody>
        <tr>
          <th scope="row"> {{ product.getId() }} </th>
          <td> {{ product.getName() }} </td>
          <td> {{ product.getProductType().getName() }}</td>
        </tr>
      </tbody>
    </table>
    <br>
    <form action="{{ path('product_delete', {'id': product.getId() }) }}" method="post">
      <input type="hidden" name="_token" value="{{ csrf_token('delete' ~ product.getId() ) }}">
      <button type="submit" class="btn btn-primary">Delete</button>
    </form>
    <br>
    <button class="btn btn-primary" onclick="window.history.back();">Go Back</button>
  </div>
</div>
```

Prakticky

```
{% block body %}

  {{ product.getId()}}<br>
  {{ product.getName()}}<br>
  <form action="{{ path('product_delete', {id:product.getId()}) }}"
  method="post">
    <input type="hidden" name="_token" value="{{ csrf_token('delete'
~product.getId() ) }}" ><br>
    <button type="submit" > Delete </button><br>
  </form>

  <button onclick="window.history.back();" > zpet </button>

{% endblock %}
```

index.html.twig

```
Smazano
{{ controllername }}
{{ variable_test }}<br>
```

X. Generování PDF:

- **composer require dompdf/dompdf**
- tvorba funkce na generování pdf

```
#[Route('/print', name: 'app_print')]
public function printPDF(ManagerRegistry $doctrine): Response
{
    $products = $doctrine->getManager()->getRepository(Product::class)->findAll(); //Najde všechny záznamy entity

    $productTypes = $doctrine->getRepository(ProductType::class)->findAll();

    foreach ($products as $product) // Spojí ProductType s Product na základě cizího klíče v Product
    {
        $type = $product->getProductType();
        foreach ($productTypes as $productType)
        {
            if ($type->getId() == $productType->getId())
            {
                $product->setProductType($productType);
                break;
            }
        }
    }

    $html = $this->renderView('/home/pdf_print.html.twig', ['products' => $products]); //Pošle záznamy pro vykreslení na html

    $dompdf = new Dompdf();
    $dompdf->loadHtml($html); //Nahraje data pro generování pdf
    $dompdf->render(); //Vykreslí podle dat
    $pdfOutput = $dompdf->output(); // Hotová stránka

    return new Response($pdfOutput, Response::HTTP_OK, ['Content-Type' => 'application/pdf']); //Předání dat a nastavení typu
}
```

- tvorba html stránky pdf_print.html.twig

```
{% block body %}
<style>
    .example-wrapper { margin: 1em auto; max-width: 800px; width: 95%; font: 18px/1.5 sans-serif; }
    .example-wrapper code { background: #F5F5F5; padding: 2px 6px; }
</style>

<div class="example-wrapper">
    <table class="table table-striped table-hover" style="width:75%; margin-left: auto; margin-right: auto;" >
        <thead class="shadow-sm">
            <tr>
                <th>Id</th>
                <th>Název</th>
                <th>Type</th>
            </tr>
        </thead>
        <tbody>
            {% for product in products %}
            <tr>
                <td> {{product.getId()}} </td>
                <td> {{product.getName()}} </td>
                <td> {{product.getProductType().getName()}}</td>
            </tr>
            {% endfor %}
        </tbody>
    </table>
</div>
{% endblock %}
```

X. Tvorba přihlášení:

- **composer require symfony/security-bundle**
- symfony console make:user User
- symfony console make:migration
- symfony console doctrine:migration:migrate
- symfony console make:registration-form (nejspíš nechcete potvrzování emailem)
- Provést registraci přes webovou stránku **/register**
- symfony console make:auth (1)
- V login controlleru nastavte přesměrování

```
public function onAuthenticationSuccess(Request $request, TokenInterface $token, string $firewallName): ?Response
{
    if ($targetPath = $this->getTargetPath($request->getSession(), $firewallName)) {
        return new RedirectResponse($targetPath);
    }

    // For example:
    // return new RedirectResponse($this->urlGenerator->generate('some_route'));
    return new RedirectResponse($this->urlGenerator->generate('app_home'));
}
```

- Teď můžete přes podobné příkazy upravit přístup k elementům

```
</li>
{% if app.user %}
<li class="nav-item">
```